

General Notes:

Website: https://fbia.yashsurjuse.dev	Slideshow: https://fbia.yashsurjuse.dev/slides
---	---

Coded in Typescript/JavaScript using React Framework

Made in Visual Studio Code

Deployed to the internet using Cloudflare

Used GitHub as version control

Optimized to be faster using React (virtual memory management, component-based, which is faster, optimized rendering with Turbopack)

Learned coding on Codecademy, Kaggle, and YouTube

Our website (fbia.yashsurjuse.dev)	Official MET Museum Website (metmuseum.org)
Fully loaded: ~430 MB of RAM	Fully loaded: ~610 MB of RAM
Avg. per page: ~162 MB of RAM	Avg. per page: ~180 MB of RAM

Researched by looking at other museum sites (mostly MET) and identifying needed pages and what content to place

Used sources for layout tips, coding help, content ideas, design suggestions, and website content

Split roles based on skill (Yash is good at coding and UX/UI, while Guriq is better at planning and designing)

Chose React+TS because ease of use (debugging), customizability, stability, and many libraries at our disposal.

31 real pages (excluding repetitive pages/paths)

All sources listed here: <https://fbia.yashsurjuse.dev/sources>

Overview of everything done + more on slideshow

No backend, all locally using LocalStorage

Sign-in/Sign-up is a mock and doesn't register in any database

Did we use outside help: No outside figure assisting in development or information; people only provided feedback. We used no AI, aside from writing some filler content.

Technical Details:

Core Stack

Framework: Next.js 16.1.6 (App Router, static export via output: "export")
Language: TypeScript 5.x (strict mode)
UI Library: React 19.2.3 with React Compiler (babel-plugin-react-compiler)
Styling: Tailwind CSS v4 (using @tailwindcss/postcss, no config file needed)
Package Manager: npm

Animation & Interaction

Framer Motion v12.35 - modals, staggered entrances, page transitions
Lenis v1.3.18 - smooth scrolling
OGL v1.0.11 - WebGL/3D graphics (Particles, Prism)
Lucide React v1.21 - icon library

State Management

Zustand v5.0.14 - global state (language preferences)
React Context - Cart/Wishlist/Orders system
localStorage - cart, wishlist, orders, and accessibility settings all persist

Accessibility

Custom-built panel with 17+ settings (no external a11y library)
Smart contrast detection using elementsFromPoint() with DOM luminance sampling
Speech Synthesis API for text-to-speech

Datasets

<i>File</i>	<i>Records</i>	<i>Size</i>
artifacts_master.json	2,932 artifacts	1,862 KB
artists_master.json	1,360 artists	560 KB
exhibitions_master.json	1,795 exhibitions	1,223 KB
store_master.json	696 products	401 KB

Our data allows for many thousands/millions of entries per set, but for optimiziblity we chose to limit
All data is client-side JSON (static export) | No backend server
Departments also fetched live from Met's public API (collectionapi.metmuseum.org) with rate limiting (80 req/s) and sessionStorage caching

Shadcn / Radix

Only one Shadcn component: Button (uses @radix-ui/react-slot for asChild pattern)
The rest of the UI is custom-built

Build & Deploy

ESLint v9 with eslint-config-next (core-web-vitals + TypeScript)

Playwright for testing

Cloudflare deployment via Wrangler (wrangler deploy)

Static export with trailingSlash: true

Scripts - 30+ Node.js data-processing scripts in /scripts for scraping, cleaning, and generating the JSON datasets from the Met API

Our Process:

1. Pre-planning
 - a. Decided on the MET museum
 - b. Imagined how the site will work/function/look
 - c. Make a few early mockups in Figma
2. Decided on technical stack
 - a. Decided what stack (config for coding) we would use (settled on React)
 - b. Planned out how we were going to handle data
 - c. Continued planning looks in Figma
3. Planned out the full setup of the site
4. Started coding early versions
 - a. Made a few designs in code and iterations
 - b. Set up an environment for mass data harvesting
 - c. Put the layout into code
5. Finding content to put on pages
 - a. Gathered many sites and generated content for all the pages
 - b. Used Official MET for some info/text
6. Perfecting the structure
 - a. Overhauled the paths/pages structured for the best UX
 - b. Got automated scraping + API fetching to export data
 - c. Added content to pages
7. UI overhaul
 - a. Was recommended to overhaul UI
 - b. Changed UI to look modern/clean
 - c. Added animations
8. Bug fixing
 - a. Fixed UI bugs, loading bugs, anomalies, etc
 - b. Changed up UI one last time globally
9. Deploying to the internet
 - a. Attempted to deploy to Cloudflare Pages
 - b. Fixed Wrangler bugs
 - c. Deployed to Cloudflare Pages from GitHub